

Beyond Code Coverage: Comparing AI-Generated Tests with Existing Human Test Filters Using Mutation Analysis

Premal Mistry, Randhir Kumar
Independent Researcher

Abstract—Code coverage does not directly measure fault-detection capability, yet it remains a common proxy for evaluating LLM-generated unit tests. We present fifteen paired component-level experiments on five open-source Swift repositories: ten exploratory (E1–E10) and five follow-up experiments (E11–E15) under an internally pre-declared neutral selection rule. For each component we compare an existing, researcher-selected Human test filter against a freshly generated, implementation-aware AI suite (Grok 4.5 via Cursor), using `llvm-cov` structural coverage and mutation testing on a shared frozen mutant set. Across both phases, suites with identical or near-identical coverage can differ in mutation score; in one case the suite with higher line coverage has the lower mutation score. We conclude that line, region, and function coverage are insufficient on their own to determine relative mutation-based effectiveness. Mutation selection was not fully blind: a retrospective audit found suite-aware rationale in 29 of 369 planned mutants (7.9%), concentrated especially in E2 and E11.

Index Terms—large language models, unit testing, mutation testing, code coverage, Swift

I. Introduction

Unit testing remains central to detecting regressions. LLMs can now synthesize executable tests from production code, making automated generation practical, but prior work reports compilation failures, incorrect assertions, and weak fault detection alongside promising coverage [1]. Structural coverage answers whether code executed; it does not by itself establish assertion quality or fault-detection power.

This study does not estimate whether AI-generated tests are better than human-written tests under equal effort. It compares two concrete artifacts per component: an existing Human test filter—a researcher-selected subset of already-written repository tests—against an independently generated AI suite—written fresh against the current implementation and iteratively repaired until it compiled and passed, without a fixed iteration, token, or time budget. The protocol therefore cannot answer the counterfactual in which humans receive matched fresh effort, or AI generation is one-pass and budget-capped.

We ask two questions. RQ1: How did mutation effectiveness differ between the selected existing Human test filters and independently generated AI suites under this study protocol? RQ2 (primary): Can line, region, and

function coverage determine their relative mutation-based effectiveness?

The study has two phases. An exploratory phase (E1–E10) established initial observations, including coverage/mutation dissociation, but later components were partly chosen for their likely ability to distinguish the suites. A neutral-selection follow-up (E11–E15) then selected components by an internally pre-declared eligibility-and-alphabetical-first rule to reduce (not eliminate) that selection discretion. Phases are reported separately throughout.

Contributions. (1) Paired, component-level comparison of an existing Human filter and an independently generated AI suite on the same production component and frozen mutant set, with leakage controls. (2) Fifteen case studies across five Swift repositories in exploratory and neutral-selection phases, centered on coverage/mutation dissociation. (3) A quantified mutation-selection look-ahead audit and an E11 sensitivity check on already-frozen results.

Artifacts (mutation plans, fingerprints, look-ahead audit, sensitivity derivation, candidate inventories) are at <https://github.com/premalmistry/beyond-code-coverage-ai-vs-human-tests>.

II. Related Work

Coverage is an imperfect adequacy proxy for human-written suites: once suite size is controlled, correlation with mutation-based effectiveness is only low to moderate [2]. Mutation testing more broadly is surveyed by Papadakis et al. [3].

For LLM-generated tests, Yuan et al. [1], Yang et al. [4], and Ouédraogo et al. [5] report strong sensitivity of compilability, coverage, and related outcomes to model and prompt choice. MuTAP [6] and MutGen [7] use surviving mutants as generation feedback; we withhold mutation information during generation and use mutation only for post-generation comparative evaluation.

Closest comparisons include Lops et al. [8] (aggregate LLM-vs-human Java suites on coverage and mutation), Vathana et al. [9] (coverage parity with a large AI-favoring real-bug gap on BugsInPy), and Zhao et al. [10] (coverage/mutation/size relationships for LLM suites). We

add a paired component-level design that makes coverage/mutation dissociation directly observable, including Human-favoring gaps under coverage parity, plus an internally pre-declared neutral follow-up and a quantified look-ahead audit of manually designed mutants. We do not claim to be first to study coverage/mutation limits.

III. Methodology

A. Subjects

We study fifteen production components from five Swift repositories (Table I). Exploratory subjects (E1–E10) required existing human tests, deterministic SwiftPM execution, and isolatable mutation potential; later exploratory components were additionally screened for diversity and for being likely to distinguish the suites. Follow-up subjects (E11–E15) were chosen by the neutral rule below and do not reuse exploratory components.

B. Human Filter and AI Suite

For each component, a focused existing Human test filter—a researcher-selected subset of already-existing tests, not a newly written suite—was frozen before AI generation. Line, region, and function coverage (llvm-cov) were recorded for this filter alone. Because the filter is a selected subset, reported Human mutation scores describe that filter, not necessarily every repository test that touches the component (e.g., related suites left out of scope in E11; a non-deterministic test excluded in E15 for deterministic baselines).

The AI suite could inspect the production implementation and APIs needed for compilation, but not human tests, fixtures, human coverage reports, mutation plans, or mutation results. Generation used Grok 4.5 through Cursor (version 3.15.19) with vendor-default sampling. Suites were iteratively corrected against compiler errors and their own failures until passing, then frozen and SHA-256-fingerprinted. This loop never consulted human tests or mutation information, but it means the AI artifact is fresher and iteratively self-corrected rather than one-shot. AI suites were larger than the Human filter in nine of ten exploratory experiments and in all five follow-up experiments.

Contamination controls used qualified test filters and, in later experiments, automated logging of executed test names to keep Human and AI executions disjoint. Production checksums were re-verified between mutants; suites were not edited after mutation outcomes were observed. Detailed fingerprints and executed-test inventories are in the artifact repository.

C. Mutation Design and Look-Ahead Disclosure

Mutation plans were defined only after both suites were frozen. Mutants represented realistic defect classes (branch inversions, comparison errors, off-by-one boundaries, wrong returns, omitted guards, sign errors, special-case handling). The same mutant was executed against

both frozen suites, with production code restored between mutants.

A mutant was killed when a test failed (including crash/timeout kills). Survivors were excluded from the denominator as equivalent only after an explicit code-level argument that no test could distinguish the mutant under the public API; adjudication was by the same person who designed prompts and plans, with no second annotator. Mutation score:

$$MS = \frac{\text{killed}}{\text{killed} + \text{survived}} \times 100. \quad (1)$$

Look-ahead disclosure. Freezing suites before mutation design prevents test-side adaptation to known mutants, but does not make mutant selection blind: the designer had access to both frozen suites’ content and coverage while choosing mutants. A retrospective audit of all 369 planned mutants found explicit suite-content-specific outcome predictions or rationales in 29 (7.9%), concentrated especially in E2 and E11. Mutation-score gaps in affected experiments are therefore conditional on suite-aware mutant sets. The full audit is in the artifact repository.

D. Neutral-Selection Follow-Up Rule

For E11–E15 we applied an internally pre-declared neutral selection rule (written and frozen before E11 outcomes; not externally pre-registered): enumerate Sources/.swift files; apply fixed eligibility criteria (existing deterministic human tests; focused observable behavior; mutation-suitable; no network; not previously studied; reasonably isolated); sort alphabetically by path; select the first eligible component; freeze it; accept the result regardless of outcome. The rule reduces researcher discretion in which component is studied; it does not randomize the sample or address mutation look-ahead.

IV. Results

All statistics characterize the observed fifteen case studies; they are not estimates of a broader population effect. We report descriptive summaries only and do not pool exploratory and follow-up win counts.

A. Mutation Scores (RQ1, Descriptive)

Exploratory (E1–E10): AI higher in six, Human in three, one tie. Experiment-weighted mean MS was 93.8% (AI) and 88.8% (Human); median paired difference (AI–Human) +8.5 points (range −8.0 to +21.8). Mutant-weighted pooled scores over 239 valid mutants: 93.31% AI vs. 88.70% Human (+4.60 points).

Follow-up (E11–E15): AI higher in all five. Mean MS 85.3% (AI) vs. 69.2% (Human); mean paired difference +16.1 points (median +8.7; range +4.5 to +50.0). Pooled over 115 valid mutants: 85.22% AI vs. 68.70% Human (+16.52 points). These descriptive follow-up results must not be read as evidence of universal AI superiority: $n=5$,

TABLE I
Study subjects.

Exp.	Phase	Repository	Component
E1	Expl.	swift-snapshot-testing	URLRequest
E2	Expl.	swift-snapshot-testing	Any.swift
E3	Expl.	swift-collections	Heap
E4	Expl.	swift-collections	OrderedSet
E5	Expl.	swift-algorithms	Combinations
E6	Expl.	swift-algorithms	Partition
E7	Expl.	swift-parsing	Prefix
E8	Expl.	swift-parsing	Digits
E9	Expl.	swift-numeric	SaturatingArithmetic
E10	Expl.	swift-numeric	Polar
E11	Follow-up	swift-snapshot-testing	SnapshotTestingConfiguration.swift
E12	Follow-up	swift-collections	RigidArray+Append.swift
E13	Follow-up	swift-algorithms	AdjacentPairs.swift
E14	Follow-up	swift-parsing	OneOfBuilder.swift
E15	Follow-up	swift-numeric	Complex+AlgebraicField.swift

TABLE II
Mutation scores for E1–E15.

Exp.	Phase	Valid	Human MS	AI MS	Higher
E1	Expl.	20	85.0%	95.0%	AI
E2	Expl.	27	70.4%	85.2%	AI
E3	Expl.	28	92.9%	85.7%	Human
E4	Expl.	25	96.0%	88.0%	Human
E5	Expl.	21	100.0%	100.0%	Tie
E6	Expl.	22	90.9%	100.0%	AI
E7	Expl.	23	91.3%	100.0%	AI
E8	Expl.	24	91.7%	100.0%	AI
E9	Expl.	26	96.2%	88.5%	Human
E10	Expl.	23	73.9%	95.7%	AI
E11	Follow-up	24	41.7%	91.7%	AI
E12	Follow-up	22	90.9%	95.5%	AI
E13	Follow-up	23	91.3%	100.0%	AI
E14	Follow-up	22	68.2%	72.7%	AI
E15	Follow-up	24	54.2%	66.7%	AI

heterogeneous manually constructed mutant sets, and look-ahead exposure (especially E11) qualify every gap.

E11 sensitivity. E11 has both the largest gap (+50.0 pp) and the highest look-ahead exposure (12/24 mutants flagged, 50.0%). Excluding E11 leaves direction unchanged (AI higher in 4/4) but roughly halves magnitude: mean paired difference +**7.58** pp (vs. +16.1) and pooled difference +**7.70** pp (vs. +16.52). Derivation details are in the artifact repository.

B. Coverage–Mutation Dissociation (RQ2)

RQ2 is the central finding. Its evidentiary base is E3, E4, E9, E13, E14, and E15—not E11. Table III summarizes the cases.

Exploratory. In E3, identical primary line coverage coincides with a 7.2-point Human-favoring MS gap; both actual Human-only kills are among flagged mutants, so the magnitude is conditioned even though coverage identity is a direct measurement. E4 shows identical line, region,

and function coverage with Human 96.0% vs. AI 88.0% MS and zero flagged mutants. E9 shows 100% line, region, and function coverage for both suites with Human 96.2% vs. AI 88.5% and zero flags—the strongest coverage-parity instance. E5 is the counterexample: near-complete identical coverage (99.21%/96.00%/94.74%) with both suites killing all 21 valid mutants, so coverage parity does not imply that an MS gap must exist.

Follow-up. E13 recurs identical line coverage (92.95%) with an 8.7-point AI-favoring gap under neutral selection; that entire gap corresponds to two suite-aware mutants (2/23), so magnitude is conditioned. E14 shows a large AI line-coverage advantage (~32 pp) but only a 4.5-point MS gap (0 flags)—coverage-gap magnitude does not reliably predict MS-gap magnitude. E15 shows an inverse line-coverage ranking: Human line coverage is substantially higher (71.19% vs. 44.92%), yet Human MS is lower (54.2% vs. 66.7%). Region and function coverage both favor AI (78.12% vs. 40.62%; 66.67% vs. 25.00%), consistent with AI’s higher MS—so it is line coverage specifically, not every structural metric, that misranks the suites. E15 has zero flagged mutants.

Narrow claim. The central RQ2 pattern does not depend on E11. It recurs when restricted to the four zero-look-ahead experiments (E4, E9, E14, E15); E3 and E13 corroborate qualitatively but their gap magnitudes are conditioned. Across these case studies, line, region, and function coverage, as measured by `llvm-cov`, are insufficient on their own to determine relative mutation-based test effectiveness.

C. Complementary Kills (Brief)

The hypothetical union of both suites’ kills exceeded the stronger single suite in 1 of 10 exploratory experiments and 3 of 5 follow-up experiments—additive detection value for these frozen artifacts, not causal synergy or evidence that the suites were executed together.

TABLE III
Coverage–mutation dissociation cases.

Exp.	Phase	Coverage relationship	Human MS	AI MS	Look-ahead
E3	Expl.	Identical line (99.03%)	92.9%	85.7%	Cond. (3/28)
E4	Expl.	Identical L/R/F (82.39%/78.85%/73.08%)	96.0%	88.0%	0
E9	Expl.	100% L/R/F both suites	96.2%	88.5%	0
E13	Follow-up	Identical line (92.95%)	91.3%	100.0%	Cond. (2/23; entire gap)
E14	Follow-up	Large line gap (~32 pp; AI 79.76% vs. Human 47.62%)	68.2%	72.7%	0
E15	Follow-up	Inverse line ranking (Human 71.19% vs. AI 44.92%); AI higher region (78.12% vs. 40.62%) and function (66.67% vs. 25.00%)	54.2%	66.7%	0

V. Threats to Validity

Effort/freshness asymmetry. AI suites are freshly generated and iteratively self-corrected; Human filters are historically written and unrevised for this study. AI-favoring mutation gaps may partly or wholly reflect authoring circumstances rather than generating-agent identity.

Researcher-selected Human filters. Scores describe selected filters, not every existing test for a component.

Mutation-selection look-ahead. Designer access to frozen suite content yielded suite-aware rationale in 29/369 (7.9%) planned mutants, concentrated in E2 and E11; affected gaps are conditional. RQ2’s core pattern still holds on zero-exposure experiments.

Manually designed mutants / single adjudicator. Heterogeneous mutant sets; equivalence judgment by the same person who designed prompts and plans; exploratory exclusion rates 0–19%.

Single LLM/tooling. Grok 4.5 via Cursor 3.15.19 with default sampling; not generalized to other models or agents.

Scope. Fifteen components from five open-source Swift repositories; exploratory selection not representative; the neutral rule reduces but does not eliminate discretion (small $n=5$, inherited repositories, researcher-designed eligibility). No claim extends to other languages or test levels.

Mutation $\neq$ real faults. Findings concern seeded syntactic mutants, not historical production defects; we make no claim that higher mutation score implies catching more real bugs.

VI. Conclusion

Across these case studies, llvm-cov line, region, and function coverage were insufficient on their own to determine relative mutation-based effectiveness. That conclusion is carried by E3, E4, E9, E13, E14, and E15—including four zero-look-ahead experiments—and does not depend on E11. Human filters and AI suites each killed mutants the other missed in some experiments; observed AI-favoring mutation advantages are conditional on this

study’s asymmetric authoring regime and, in places, on suite-aware mutant selection. We do not claim general AI superiority, and mutation results do not establish real-bug superiority.

Future work should use blinded or systematically sampled mutant selection, matched-effort Human/AI comparisons, additional languages and models, and real-fault corpora.

References

- [1] Z. Yuan, M. Liu, S. Ding, K. Wang, Y. Chen, X. Peng, and Y. Lou, “No more manual tests? evaluating and improving chatgpt for unit test generation,” *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, 2024, originally released as arXiv:2305.04207 (2023).
- [2] L. Inozemtseva and R. Holmes, “Coverage is not strongly correlated with test suite effectiveness,” in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, 2014, pp. 435–445.
- [3] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman, “Mutation testing advances: An analysis and survey,” *Advances in Computers*, vol. 112, pp. 275–378, 2019.
- [4] L. Yang, C. Yang, S. Gao, W. Wang, B. Wang, Q. Zhu, X. Chu, J. Zhou, G. Liang, Q. Wang, and J. Chen, “On the evaluation of large language models in unit test generation,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024.
- [5] W. C. Ouedraogo, A. K. Kaboré, Y. Li, H. Tian, A. Koyuncu, J. Klein, D. Lo, and T. F. Bissyandé, “Prompt engineering in llms for automated unit test generation: A large-scale study,” *Empirical Software Engineering*, vol. 31, no. 4, 2026, originally released as arXiv:2407.00225, 2024.
- [6] A. Moradi Dakhel, A. Nikanjam, V. Majdinasab, F. Khomh, and M. C. Desmarais, “Effective test generation using pre-trained large language models and mutation testing,” *Information and Software Technology*, vol. 171, p. 107468, 2024.
- [7] G. Wang, Q. Xu, L. Briand, and K. Liu, “Mutation-guided unit test generation with a large language model,” *IEEE Transactions on Software Engineering*, vol. 52, no. 5, pp. 1657–1671, 2026.
- [8] A. Lops, F. Narducci, A. Ragone, M. Trizio, and C. Bartolini, “Llms for automated unit test generation and assessment in java: The agonetest framework,” in *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025, pp. 2401–2413, originally released as arXiv:2511.20403, 2025.
- [9] P. Vathana, P. Bhatt, R. Patel, and N. U. Eisty, “Llm vs. human unit tests: Fault detection on real python bugs,” arXiv preprint arXiv:2606.08588, 2026. [Online]. Available: <https://arxiv.org/abs/2606.08588>

- [10] J. Zhao, S. Zhou, and E. Cohen, “Do coverage and mutation scores of llm-generated test suites correlate with their effectiveness? (replicability study),” arXiv preprint arXiv:2607.22880, 2026. [Online]. Available: <https://arxiv.org/abs/2607.22880>